

Multimodal Visual Understanding Application

Multimodal Visual Understanding Application

1. Concept Introduction
 - 1.1 What is "Visual Understanding"?
 - 1.2 Implementation Principles
2. Code Analysis
 - Key Code
 1. Tool Layer Entry (`largetmodel/utis/tools_manager.py`)
 2. Model Interface Layer (`largetmodel/utis/large_model_interface.py`)
 - Code Analysis
3. Practical Operations.
 - 3.1 Configuring the Offline Large Model
 - 3.1.1 Configuring the LLM Platform (`yahboom.yaml`)
 - 3.1.2 Configuring the Model Interface (`large_model_interface.yaml`)
 - 3.1.3 How to Use a CSI Camera
 - 3.2 Starting and Testing the Function (Text Input Mode)
4. Common Problems and Solutions
 - Problem 1: The log displays "Failed to call ollama vision model" or the connection is refused.
 - Problem 2: The `seewhat` tool returns "Unable to open camera" or fails to take a photo.

Note: Due to performance limitations, the Raspberry Pi 5 1GB, 2GB, and 4GB versions cannot run offline. You can refer to online large models.

1. Concept Introduction

1.1 What is "Visual Understanding"?

In the `largetmodel` project, the **multimodal visual understanding** feature enables robots to go beyond simply "seeing" a matrix of pixels and truly "understand" the content, objects, scenes, and relationships within an image. This is like giving robots a pair of thinking eyes.

The core tool of this feature is `seewhat`. When a user issues a command such as "see what's here," the system invokes this tool, triggering a series of background operations that ultimately provide the user with natural language feedback on the AI's analysis of the real-time image.

1.2 Implementation Principles

The basic principle is to feed two different types of information—**images (visual information)** and **text (linguistic information)**—into a powerful multimodal large model (such as LLaVA).

1. **Image Encoding:** The model first uses a visual encoder to convert the input image into a computer-interpretable numerical vector that captures features such as color, shape, and texture.
2. **Text Encoding:** Simultaneously, the user's question (e.g., "What's on the table?") is also converted into a text vector.

3. **Cross-modal Fusion:** The most critical step is to fuse the image and text vectors in a special "attention layer." Here, the model learns to "focus" on the parts of the image relevant to the question. For example, when asked about the word "table," the model will pay more attention to areas of the image that match the characteristics of a table.
4. **Answer Generation:** Finally, a large language model (LLM) generates a descriptive text answer based on this fused information.

Simply put, **it uses text to highlight corresponding parts of the image, and then describes the highlighted parts verbally.**

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utis/tools_manager.py`)

The `seewhat` function in this file defines the tool's execution flow.

```
# From largemodel/utis/tools_manager.py

class ToolsManager:
    # ...

    def seewhat(self):
        """
        Capture camera frame and analyze environment with AI model.

        :return: Dictionary with scene description and image path, or None if
        failed.
        """
        self.node.get_logger().info("Executing seewhat() tool")
        image_path = self.capture_frame()
        if image_path:
            # Use isolated context for image analysis.
            analysis_text = self._get_actual_scene_description(image_path)

            # Return structured data for the tool chain.
            return {
                "description": analysis_text,
                "image_path": image_path
            }
        else:
            # ... (Error handling)
            return None

    def _get_actual_scene_description(self, image_path, message_context=None):
        """
        Get AI-generated scene description for captured image.

        :param image_path: Path to captured image file.
        :return: Plain text description of scene.
        """
```

```

try:
    # ... (Building Prompt)
    result = self.node.model_client.infer_with_image(image_path,
scene_prompt, message=simple_context)
    # ... (Processing results)
    return description
except Exception as e:
    # ...

```

2. Model Interface Layer

(`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all image understanding tasks. It is responsible for calling the specific model implementation based on the configuration.

```

# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface. """
        # ... (Prepare Message)
        try:
            # Determine which specific implementation to call based on the value
            # of self.llm_platform
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... Logic for calling the Tongyi model
                pass
            # ... (Logic of other platforms)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}

```

Code Analysis

This feature's implementation involves two main layers: the tool layer defines the business logic, and the model interface layer is responsible for communicating with the large language model. This layered design is key to achieving platform versatility.

1. Tool Layer (`tools_manager.py`):

- The `seewhat` function is the core business of the visual understanding function. It encapsulates the entire "seeing" action process: first, it calls the `capture_frame` method to obtain an image, then calls `_get_actual_scene_description` to prepare a prompt for the model to analyze the image.
- The most critical step is calling the `infer_with_image` method of the model interface layer. It does not care about the underlying model; it only passes the two core data elements, "image" and "analysis instructions," to the model interface layer.

- Finally, it packages the analysis results (plain text descriptions) received from the model interface layer into a structured dictionary and returns them. This allows upper-layer applications to easily use the analysis results.

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_image` function acts as a "dispatching center." Its primary responsibility is to check the current platform configuration (`self.llm_platform`) and, based on the configuration, dispatch tasks to specific handlers (such as `ollama_infer` or `tongyi_infer`).
- This layer is key to adapting to different AI platforms. All platform-specific operations (such as data encoding and API call formats) are encapsulated within their respective handlers.
- In this way, the business logic code in `tools_manager.py` remains unchanged to support a variety of different large-model backend services. It simply interacts with the unified, stable `infer_with_image` interface.

In summary, the `seewhat` tool's execution flow demonstrates a clear separation of responsibilities: `ToolsManager` defines the "what" (acquiring an image and requesting analysis), while `model_interface` defines the "how" (selecting the appropriate model platform based on the current configuration and interacting with it). This makes the tutorial's parsing universal; the core code logic remains consistent regardless of whether the user is using the app in online or offline mode.

3. Practical Operations.

3.1 Configuring the Offline Large Model

3.1.1 Configuring the LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its primary language model.

First, enter the Docker container by entering the command in the terminal:

```
./ros2_docker.sh
```

If you need to enter other commands in the same Docker container later, simply enter `./ros2_docker.sh` again in the host terminal.

1. Open the file in the terminal:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. Modify/Confirm `llm_platform`:

```

model_service:                                     #Model server node parameters
  ros__parameters:
    language: 'zh'                                #Large Model Interface Language
    useolinetts: True                             #This item is invalid in text mode
    and can be ignored

    # Large model configuration
    llm_platform: 'ollama'                        # KEY: Make sure this is 'ollama'
    regional_setting : "China"
    camera_type: 'csi'                            # Camera type: 'csi', 'usb'
    mjpeg_stream_url: 'http://172.17.0.1:8080/camera.mjpg' # MJPEG stream URL
    for CSI camera in Docker

```

3.1.2 Configuring the Model Interface (large_model_interface.yaml)

This file defines which visual model to use when the `ollama` platform is selected.

1. Open the file in a terminal.

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2. Find the ollama related configuration

```

#.....
## Offline Large Language Models
# Ollama Configuration
ollama_host: "http://localhost:11434" # ollama server address
ollama_model: "llava" # Key: Replace this with the multimodal model you
downloaded, such as "llava"
#.....

```

Note: Please ensure that the model specified in the configuration parameters (e.g., `llava`) can handle multimodal input.

3.1.3 How to Use a CSI Camera

First, ensure that `camera_type` in `yahboom.yaml` is set to CSI.

Then connect the CSI camera to the J4 port on the motherboard. Then, enter the following command in the host terminal:

```
python ~/host_stream.py
```

```

[0:02:08.719748357] [2401] INFO Camera camera_manager.cpp:326 libcamera v0.5.0+59-d83ff0a4
[0:02:08.727314252] [2420] INFO RPI pisp.cpp:720 libpisp version v1.2.1 981977ff21f3 29-04-2025 (14:13:50)
[0:02:08.736599523] [2420] INFO RPI pisp.cpp:1179 Registered camera /base/axi/pcie@1000120000/rp1/i2c@800000/imx219@10 to CFE
device /dev/media0 and ISP device /dev/media1 using PiSP variant BCM2712_D0
[0:02:08.739514459] [2401] INFO Camera camera.cpp:1205 configuring streams: (0) 640x480-XBGR8888 (1) 640x480-BGGR_PISP_COMP1
[0:02:08.739627366] [2420] INFO RPI pisp.cpp:1483 Sensor: /base/axi/pcie@1000120000/rp1/i2c@800000/imx219@10 - Selected sensor
format: 640x480-SBGGR10_1X10 - Selected CFE format: 640x480-PC1B
* Serving Flask app 'host_stream'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://192.168.2.26:8080
Press CTRL+C to quit

```

After successful activation, you can proceed with the following steps.

3.2 Starting and Testing the Function (Text Input Mode)

Note: The performance will be poor when using a model with a small number of parameters or when running at a very low memory usage. For a better experience with this function, please refer to the corresponding section in <Online Large Model (Text Interaction)>.

1. **Start the `largemodel` main program (text mode):**

Open a terminal and enter Docker, then run the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
```

2. **Send a text command:**

Open another new terminal, enter the same Docker container, and run the following command:

```
ros2 run text_chat text_chat
```

Then start typing: "What do you see?"

3. **Observation:**

In the first terminal where the main program is run, you will see log output indicating that the system received the text command, called the `seewhat` tool, and ultimately printed out a text description of the desktop generated by the LLaVA model.

4. Common Problems and Solutions

Problem 1: The log displays "Failed to call ollama vision model" or the connection is refused.

Solution:

1. **Check the Ollama service:** Run `ollama list` in the terminal to ensure that `llava` (or the model you configured) has been downloaded and the Ollama service is running.
2. **Check the configuration files:** Carefully verify that the configuration in `yahboom.yaml` and `large_model_interface.yaml` is correct, especially the values for `llm_platform` and `ollama_model`.

Problem 2: The `seewhat` tool returns "Unable to open camera" or fails to take a photo.

Solution:

1. **Check device connections:** Run `ls /dev/video*` to ensure the system can detect your USB camera. For CSI cameras, ensure you connect them to port J4 and run `host_stream.py` on the host machine.
2. **Permission issue:** Try testing the camera using an application such as `cheese` or `gview`. If `sudo` works but a regular user doesn't, it may be a problem with udev rules or user group permissions.